HerbCheck: code listing

Repository: github.com/udaytx009/uday-shankar-herbcheck-db2 (MIT). Files included: README.md, sql/schema.sql, app/config.py, app/embeddings.py, app/repo.py, app/chain.py, app/ingest.py, app/api.py, tests/test_chain.py, tests/test_kb.py, tests/test_api.py, scripts/setup_env.sh, scripts/smoke_test.sh, docker-compose.yml. The reference data file (data/interaction_kb.json) and the web page (app/static/index.html) are in the repository.

README.md

````
# HerbCheck on IBM Db2

**Herb–drug interaction reference that uses IBM Db2 12.1 as its knowledge base, vector store and audit trail, orchestrated with La
A research demo by Uday Shankar Bhowal (LearnHive Labs). Open source (MIT). Synthetic, **unreviewed** reference data.

> **Educational reference only. Not medical advice, not a medical device.** The reference set (`data/interaction_kb.json`, v0.1.0-

## Problem

People mix herbal products (ginkgo, St John's wort, ginger, turmeric...) with prescription drugs, and clinically relevant interact

## What Db2 does here (the core of the project)

| Db2 capability | How it is used |
|---|---|
| Relational model, constraints, FK cascade, CHECK, index | `HERBS`, `DRUGS`, `DRUG_CLASSES`, `INTERACTION_RULES`, `RULE_CLASSES`,
| SQL joins | One join returns every rule of a herb that applies to the drug classes of all the user's medications (`Db2Repo.rules
| **`VECTOR(384, FLOAT32)` columns + `VECTOR_DISTANCE(..., COSINE)`** | "Did you mean" for misspelled herb and drug names, compute
| **LangChain `DB2VS` vector store** (`langchain-db2`) | Evidence passages for each rule are embedded and retrieved with Db2 simil
| View with `LISTAGG` | `V_RULE_OVERVIEW` for browsing rules by herb |
| Identity column + audit table | `CHECK_LOG` records counts and a one-way digest of every check, never the raw medication or herb

## Architecture

```mermaid
flowchart LR
  UI[Browser UI / n8n webhook] -->|POST /api/check| API[FastAPI]
  API --> CH[LangChain Runnable chain]
  CH -->|1 resolve names<br/>SQL + VECTOR_DISTANCE| DB[(IBM Db2 12.1<br/>relational + VECTOR)]
  CH -->|2 interaction rules<br/>SQL join| DB
  CH -->|3 evidence retrieval<br/>DB2VS similarity search| DB
  CH -->|4 confidence gate| CH
  CH -->|5 explanation<br/>template or optional LLM| CH
  CH -->|6 audit row| DB
```

Pipeline stages (each is a `RunnableLambda`, timed and returned in the response): `normalize -> resolve_db2 -> interactions_db2_sq

**Safety design**
- A misspelled name is only *suggested* ("did you mean"), never auto-accepted.
- Status is `delivered` only if every herb and medication was recognized and every finding has confidence >= 0.80; otherwise `need
- The optional LLM can only rephrase the retrieved findings (temperature 0, "use only these facts"); it is off by default and the

## Quick start (about 10 minutes)

Works in GitHub Codespaces or any machine with Docker (about 4 GB RAM free) and Python 3.10+.

```bash
# 1. Start Db2 Community Edition (free). First run downloads ~2 GB and takes 5-10 min.
bash scripts/setup_env.sh        # creates .env with a random Db2 password
docker compose up -d
docker compose logs -f db2      # wait for "Setup has completed", then Ctrl+C

# 2. Install and load the data
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pip install -r requirements-extra.txt   # optional: real semantic embeddings (falls back automatically if it fails)
python -m app.ingest --reset

# 3. Run
uvicorn app.api:app --host 0.0.0.0 --port 8000
# open http://localhost:8000  (in Codespaces: the "Ports" tab -> open port 8000)

# 4. Test
scripts/smoke_test.sh
pip install -r requirements-dev.txt && pytest      # 17 offline tests, no Db2 needed
```

Without Docker you can point the app at any Db2 12.1.2+ (including IBM Db2 SaaS) through the `DB2_*` variables in `.env`.

## API

`POST /api/check` with `{"medications": ["warfarin"], "herbs": ["ginkgo"]}` (max 6 medications, 5 herbs, 60 characters each) retur
`GET /api/health`, `GET /api/stats` (live counts and audit totals read from Db2).

## n8n front door (optional)

`n8n/herbcheck-db2-front-door.workflow.json`: Webhook -> HTTP Request to this API -> Respond. Import it in n8n and set `HERBCHECK_
````

Repository layout

```
app/        chain.py (LangChain pipeline), repo.py (all Db2 access), ingest.py, api.py, embeddings.py, static/index.html
sql/        schema.sql
data/       interaction_kb.json (25 herbs, 21 drug classes, 160 drugs, 44 rules; unreviewed starter)
tests/      17 tests; tests/memory_repo.py swaps Db2 for an in-memory repo
n8n/        optional workflow
```

Limits and honesty

- Reference set is a small unreviewed starter, not a clinical product. It needs pharmacist review before any real-world use.
- Drugs are matched by name to a class list; brand names and combination products are mostly not covered.
- Synthetic/demo only: do not enter real patient data.
- DB2VS applies metadata filters after the top-k search, so retrieval over-fetches (k=50) and filters by herb.

Credits

IBM Db2 Community Edition, `langchain-db2`, LangChain, FastAPI, optional FastEmbed (BAAI/bge-small-en-v1.5). MIT license.

sql/schema.sql

```
-- IBM Db2 12.1.x schema for HerbCheck on Db2. Synthetic, unreviewed reference data only.
-- Statements end with a semicolon at the end of a line (app/ingest.py splits on that).

CREATE TABLE HERBS (
  HERB_ID   INTEGER      NOT NULL PRIMARY KEY,
  NAME      VARCHAR(80)  NOT NULL UNIQUE,
  EMBEDDING VECTOR(384, FLOAT32)
);

CREATE TABLE HERB_ALIASES (
  HERB_ID INTEGER     NOT NULL,
  ALIAS   VARCHAR(80) NOT NULL,
  PRIMARY KEY (HERB_ID, ALIAS),
  FOREIGN KEY (HERB_ID) REFERENCES HERBS (HERB_ID) ON DELETE CASCADE
);

CREATE TABLE DRUG_CLASSES (
  CLASS_ID   INTEGER     NOT NULL PRIMARY KEY,
  CLASS_NAME VARCHAR(40) NOT NULL UNIQUE
);

CREATE TABLE DRUGS (
  DRUG_ID   INTEGER     NOT NULL PRIMARY KEY,
  NAME      VARCHAR(80) NOT NULL UNIQUE,
  EMBEDDING VECTOR(384, FLOAT32)
);

CREATE TABLE DRUG_CLASS_MEMBERS (
  DRUG_ID  INTEGER NOT NULL,
  CLASS_ID INTEGER NOT NULL,
  PRIMARY KEY (DRUG_ID, CLASS_ID),
  FOREIGN KEY (DRUG_ID)  REFERENCES DRUGS (DRUG_ID)          ON DELETE CASCADE,
  FOREIGN KEY (CLASS_ID) REFERENCES DRUG_CLASSES (CLASS_ID)  ON DELETE CASCADE
);

CREATE TABLE INTERACTION_RULES (
  RULE_ID   INTEGER      NOT NULL PRIMARY KEY,
  HERB_ID   INTEGER      NOT NULL,
  ATTENTION VARCHAR(10)  NOT NULL CHECK (ATTENTION IN ('high', 'moderate', 'low')),
  EVIDENCE  VARCHAR(120) NOT NULL,
  SUMMARY   VARCHAR(600) NOT NULL,
  MECHANISM VARCHAR(300) NOT NULL,
  FOREIGN KEY (HERB_ID) REFERENCES HERBS (HERB_ID) ON DELETE CASCADE
);

CREATE INDEX IX_RULES_HERB ON INTERACTION_RULES (HERB_ID);

CREATE TABLE RULE_CLASSES (
  RULE_ID  INTEGER NOT NULL,
  CLASS_ID INTEGER NOT NULL,
  PRIMARY KEY (RULE_ID, CLASS_ID),
  FOREIGN KEY (RULE_ID)  REFERENCES INTERACTION_RULES (RULE_ID) ON DELETE CASCADE,
  FOREIGN KEY (CLASS_ID) REFERENCES DRUG_CLASSES (CLASS_ID)     ON DELETE CASCADE
);

-- Privacy-by-design audit trail: counts and a one-way digest only, never the raw inputs.
CREATE TABLE CHECK_LOG (
  LOG_ID        INTEGER     NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  TS            TIMESTAMP   NOT NULL DEFAULT CURRENT TIMESTAMP,
  N_HERBS       SMALLINT    NOT NULL,
  N_DRUGS       SMALLINT    NOT NULL,
  N_FINDINGS    SMALLINT    NOT NULL,
  MAX_ATTENTION VARCHAR(10) NOT NULL,
  STATUS        VARCHAR(12) NOT NULL,
  DIGEST        CHAR(16)    NOT NULL
);

CREATE VIEW V_RULE_OVERVIEW AS
  SELECT H.NAME AS HERB, R.ATTENTION, R.EVIDENCE, R.SUMMARY,
         (SELECT LISTAGG(C.CLASS_NAME, ', ') WITHIN GROUP (ORDER BY C.CLASS_NAME)
            FROM RULE_CLASSES RC JOIN DRUG_CLASSES C ON C.CLASS_ID = RC.CLASS_ID
           WHERE RC.RULE_ID = R.RULE_ID) AS DRUG_CLASSES
    FROM INTERACTION_RULES R JOIN HERBS H ON H.HERB_ID = R.HERB_ID;
```

app/config.py

```
"""Settings, read from environment variables (or a local .env file)."""
import os

try:  # python-dotenv is optional at runtime
    from dotenv import load_dotenv

    load_dotenv()
except Exception:  # pragma: no cover
    pass

DB2 = {
    "host": os.getenv("DB2_HOST", "localhost"),
    "port": os.getenv("DB2_PORT", "50000"),
    "database": os.getenv("DB2_DATABASE", "herbdb"),
    "user": os.getenv("DB2_USER", "db2inst1"),
    "password": os.getenv("DB2_PASSWORD", ""),
    "security": os.getenv("DB2_SECURITY", "false").lower() == "true",
}

EMBED_DIM = 384  # BAAI/bge-small-en-v1.5 and the built-in hashing embedder both produce 384 dims
EMBEDDINGS = os.getenv("EMBEDDINGS", "auto").lower()
_sd = os.getenv("SUGGEST_MAX_DIST", "").strip()
SUGGEST_MAX_DIST = float(_sd) if _sd else None  # None = use the embedder's own default
EVIDENCE_TABLE = "HERB_EVIDENCE_VS"  # table owned by langchain_db2.DB2VS

LLM_BASE_URL = os.getenv("LLM_BASE_URL", "").strip()
LLM_API_KEY = os.getenv("LLM_API_KEY", "").strip()
LLM_MODEL = os.getenv("LLM_MODEL", "").strip()

MAX_MEDICATIONS = 6
MAX_HERBS = 5
MAX_NAME_LEN = 60
```

app/embeddings.py

```
"""Embedding back-ends (LangChain `Embeddings` objects, 384 dimensions each).

* FastEmbedEmbeddings: real semantic vectors (BAAI/bge-small-en-v1.5), CPU only.
* HashEmbeddings: dependency-free fallback using hashed character 3-grams and words.
  It is deterministic, needs no download, and is good at typo matching ("gingko" ~ "ginkgo"),
  so the demo never fails because a model could not be downloaded.
"""
from __future__ import annotations

import hashlib
import math
import re

from langchain_core.embeddings import Embeddings

from . import config

DIM = config.EMBED_DIM


class HashEmbeddings(Embeddings):
    name = "hash-384"
    suggest_max_dist = 0.65  # cosine distance; typos land ~0.5-0.6, unrelated names >= 0.78

    @staticmethod
    def _vec(text: str) -> list[float]:
        t = re.sub(r"[^a-z0-9 ]", " ", text.lower())
        t = re.sub(r"\s+", " ", t).strip()
        padded = f" {t} "
        feats = [padded[i : i + 3] for i in range(len(padded) - 2)]
        feats += [f"w:{w}" for w in t.split() for _ in range(2)]
        v = [0.0] * DIM
        for f in feats:
            h = int(hashlib.md5(f.encode()).hexdigest(), 16)
            v[h % DIM] += 1.0 if (h >> 64) & 1 else -1.0
        norm = math.sqrt(sum(x * x for x in v)) or 1.0
        return [x / norm for x in v]

    def embed_documents(self, texts: list[str]) -> list[list[float]]:
        return [self._vec(t) for t in texts]

    def embed_query(self, text: str) -> list[float]:
        return self._vec(text)


class FastEmbedEmbeddings(Embeddings):
    name = "fastembed-bge-small-en-v1.5"
    suggest_max_dist = 0.30  # bge similarities are compressed high, so the cut-off is tighter

    def __init__(self) -> None:
        from fastembed import TextEmbedding  # imported lazily

        self._model = TextEmbedding("BAAI/bge-small-en-v1.5")

    def embed_documents(self, texts: list[str]) -> list[list[float]]:
        return [list(map(float, v)) for v in self._model.embed(texts)]

    def embed_query(self, text: str) -> list[float]:
        return self.embed_documents([text])[0]


def get_embeddings() -> Embeddings:
    mode = config.EMBEDDINGS
    if mode == "hash":
        return HashEmbeddings()
    try:
        return FastEmbedEmbeddings()
    except Exception as exc:  # no fastembed, no network, unsupported Python, ...
        if mode == "fastembed":
            raise
        print(f"[embeddings] fastembed unavailable ({exc.__class__.__name__}); using hashing embedder")
        return HashEmbeddings()
```

app/repo.py

```
"""All Db2 access lives here (relational queries, VECTOR_DISTANCE queries, LangChain DB2VS retrieval).

The chain in chain.py only talks to this small interface, so tests can swap in an in-memory repo.
"""
from __future__ import annotations

import threading
from typing import Any

import ibm_db_dbi

from . import config

NEAREST_TABLES = {"HERBS", "DRUGS"}  # whitelist: table names are never taken from user input


def vec_literal(vec: list[float]) -> str:
    """Render a vector as the string literal Db2's VECTOR() function accepts."""
    return "[" + ",".join(f"{float(x):.6f}" for x in vec) + "]"


def dsn() -> str:
    d = config.DB2
    s = (
        f"DATABASE={d['database']};HOSTNAME={d['host']};PORT={d['port']};PROTOCOL=TCPIP;"
        f"UID={d['user']};PWD={d['password']};"
    )
    if d["security"]:
        s += "SECURITY=SSL;"
    return s


class Db2Repo:
    def __init__(self, embeddings=None) -> None:
        self._lock = threading.RLock()
        self._conn = None
        self._vs = None
        self._vs_conn = None
        self._embeddings = embeddings
        self._version: str | None = None

    # ---- connection helpers ------------------------------------------------
    def conn(self):
        if self._conn is None:
            self._conn = ibm_db_dbi.connect(dsn(), "", "")
        return self._conn

    def _reset(self) -> None:
        try:
            if self._conn is not None:
                self._conn.close()
        except Exception:
            pass
        self._conn = None

    def _run(self, sql: str, params: tuple = (), fetch: bool = True):
        """Execute one statement; reconnect once if the connection died."""
        with self._lock:
            for attempt in (1, 2):
                try:
                    cur = self.conn().cursor()
                    try:
                        cur.execute(sql, params)
                        rows = cur.fetchall() if fetch else None
                        if not fetch:
                            self.conn().commit()
                        return rows
                    finally:
                        cur.close()
                except ibm_db_dbi.Error:
                    try:
                        self.conn().rollback()
                    except Exception:
                        pass
                    if attempt == 2:
                        raise
                    self._reset()
    # ---- metadata ----------------------------------------------------------
```

```
    def version(self) -> str:
        if self._version is None:
            row = self._run("SELECT service_level FROM TABLE(sysproc.env_get_inst_info())")[0]
            self._version = str(row[0])
        return self._version

    def stats(self) -> dict[str, Any]:
        counts = self._run(
            "SELECT (SELECT COUNT(*) FROM HERBS), (SELECT COUNT(*) FROM DRUGS), "
            "(SELECT COUNT(*) FROM INTERACTION_RULES), (SELECT COUNT(*) FROM CHECK_LOG) "
            "FROM SYSIBM.SYSDUMMY1"
        )[0]
        by_attention = self._run(
            "SELECT MAX_ATTENTION, COUNT(*) FROM CHECK_LOG GROUP BY MAX_ATTENTION ORDER BY 2 DESC"
        )
        return {
            "herbs": int(counts[0]),
            "drugs": int(counts[1]),
            "rules": int(counts[2]),
            "checks_logged": int(counts[3]),
            "checks_by_max_attention": {str(a): int(n) for a, n in by_attention},
            "db2_version": self.version(),
        }

    # ---- relational lookups ------------------------------------------------
    def find_herb(self, name: str):
        rows = self._run(
            "SELECT H.HERB_ID, H.NAME FROM HERBS H WHERE LOWER(H.NAME) = ? "
            "UNION "
            "SELECT H.HERB_ID, H.NAME FROM HERB_ALIASES A JOIN HERBS H ON H.HERB_ID = A.HERB_ID "
            "WHERE A.ALIAS = ?",
            (name, name),
        )
        return (int(rows[0][0]), str(rows[0][1])) if rows else None

    def find_drug(self, name: str):
        rows = self._run("SELECT DRUG_ID, NAME FROM DRUGS WHERE NAME = ?", (name,))
        return (int(rows[0][0]), str(rows[0][1])) if rows else None

    def drug_classes(self, drug_id: int):
        rows = self._run(
            "SELECT C.CLASS_ID, C.CLASS_NAME FROM DRUG_CLASS_MEMBERS M "
            "JOIN DRUG_CLASSES C ON C.CLASS_ID = M.CLASS_ID WHERE M.DRUG_ID = ? ORDER BY C.CLASS_NAME",
            (drug_id,),
        )
        return [(int(r[0]), str(r[1])) for r in rows]

    def rules(self, herb_id: int, class_ids: list[int]):
        """Interaction rules of one herb that apply to any of the given drug classes (one SQL join)."""
        if not class_ids:
            return []
        marks = ",".join("?" for _ in class_ids)
        rows = self._run(
            "SELECT R.RULE_ID, R.ATTENTION, R.EVIDENCE, R.SUMMARY, R.MECHANISM, C.CLASS_NAME "
            "FROM INTERACTION_RULES R "
            "JOIN RULE_CLASSES RC ON RC.RULE_ID = R.RULE_ID "
            "JOIN DRUG_CLASSES C ON C.CLASS_ID = RC.CLASS_ID "
            f"WHERE R.HERB_ID = ? AND RC.CLASS_ID IN ({marks}) ORDER BY R.RULE_ID",
            (herb_id, *class_ids),
        )
        return [
            {
                "rule_id": int(r[0]),
                "attention": str(r[1]),
                "evidence": str(r[2]),
                "summary": str(r[3]),
                "mechanism": str(r[4]),
                "class_name": str(r[5]),
            }
            for r in rows
        ]

    # ---- vector features ---------------------------------------------------
    def nearest(self, table: str, vec: list[float], k: int = 3):
        """Nearest names by cosine distance, computed inside Db2 with VECTOR_DISTANCE."""
        if table not in NEAREST_TABLES:
            raise ValueError("table not allowed")
        sql = (
            f"SELECT NAME, VECTOR_DISTANCE(EMBEDDING, VECTOR('{vec_literal(vec)}', {config.EMBED_DIM}, FLOAT32), COSINE) AS D "
            f"FROM {table} ORDER BY D FETCH FIRST {int(k)} ROWS ONLY"
        )
```

```
        return [(str(r[0]), float(r[1])) for r in self._run(sql)]

    def _vector_store(self):
        from langchain_db2 import DB2VS
        from langchain_db2.db2vs import DistanceStrategy

        with self._lock:
            if self._vs is None or self._vs_conn is not self.conn():
                self._vs = DB2VS(
                    embedding_function=self._embeddings,
                    table_name=config.EVIDENCE_TABLE,
                    client=self.conn(),
                    distance_strategy=DistanceStrategy.COSINE,
                )
                self._vs_conn = self._conn
            return self._vs

    def evidence(self, query: str, herb: str, k: int = 3):
        """LangChain retrieval over the DB2VS table (Db2 vector similarity), restricted to one herb."""
        with self._lock:
            hits = self._vector_store().similarity_search_with_score(query, k=50)
        out = []
        for doc, dist in hits:
            if doc.metadata.get("herb") == herb:
                out.append({"text": doc.page_content, "distance": round(float(dist), 4), "rule_id": doc.metadata.get("rule_id")})
            if len(out) >= k:
                break
        return out

    # ---- audit ------------------------------------------------------------
    def log_check(self, n_herbs: int, n_drugs: int, n_findings: int, max_attention: str, status: str, digest: str) -> None:
        self._run(
            "INSERT INTO CHECK_LOG (N_HERBS, N_DRUGS, N_FINDINGS, MAX_ATTENTION, STATUS, DIGEST) VALUES (?, ?, ?, ?, ?, ?)",
            (n_herbs, n_drugs, n_findings, max_attention, status, digest),
            fetch=False,
        )
```

app/chain.py

```
"""The HerbCheck pipeline, built as a LangChain Runnable sequence.

normalize -> resolve (Db2 SQL + Db2 vector "did you mean") -> interactions (Db2 SQL join)
-> evidence (LangChain DB2VS retrieval) -> gate -> explain (template, or optional LLM) -> audit (Db2)

Safety design: unknown names are only ever *suggested*, never auto-accepted; a missing record is
never reported as "safe"; findings with weak evidence send the whole result to `needs_review`.
"""
from __future__ import annotations

import hashlib
import json
import re
import time
from typing import Any

from langchain_core.runnables import RunnableLambda

from . import config

ATT_RANK = {"none": 0, "low": 1, "moderate": 2, "high": 3}
CONFIDENCE_GATE = 0.80
DISCLAIMER = (
    "Educational reference only. Not medical advice and not a medical device. The reference set is an "
    "unreviewed starter; a missing record never means a combination is safe. Ask a pharmacist or clinician."
)


def clean(text: str) -> str:
    t = re.sub(r"[^a-z0-9 .'\-]", " ", str(text).lower())
    return re.sub(r"\s+", " ", t).strip()


def confidence(evidence: str) -> float:
    """Map the evidence type of a rule to a confidence score."""
    parts = [p.strip() for p in evidence.lower().split(";") if p.strip()] or ["unknown"]
    scores = []
    for p in parts:
        if "pharmacokinetic" in p or "clinical" in p:
            scores.append(0.93)
        elif "regulatory" in p:
            scores.append(0.90)
        elif "case" in p:
            scores.append(0.85)
        elif "limited" in p or "small" in p:
            scores.append(0.80)
        elif "mixed" in p:
            scores.append(0.70)
        else:  # theoretical, in vitro, unknown
            scores.append(0.60)
    score = max(scores)
    if "mixed" in evidence.lower():
        score = min(score, 0.78)
    return round(score, 2)


def _stage(name: str, fn):
    def run(state: dict[str, Any]) -> dict[str, Any]:
        t0 = time.perf_counter()
        out = fn(state)
        state["timings_ms"][name] = round((time.perf_counter() - t0) * 1000, 1)
        return out if out is not None else state

    return RunnableLambda(run, name=name)


def build_chain(repo, embeddings, explainer=None):
    """Return a Runnable: {"medications": [...], "herbs": [...]} -> result dict."""

    def normalize(state):
        def prep(items):
            seen, out = set(), []
            for raw in items:
                c = clean(raw)
                if c and c not in seen:
                    seen.add(c)
                    out.append(c)
            return out
```

```
    state["meds_in"] = prep(state.get("medications", []))
    state["herbs_in"] = prep(state.get("herbs", []))
    return state

def resolve(state):
    herbs, drugs = [], []
    unknown = {"herbs": [], "drugs": []}
    for name in state["herbs_in"]:
        hit = repo.find_herb(name)
        if hit:
            herbs.append({"input": name, "id": hit[0], "name": hit[1]})
        else:
            unknown["herbs"].append({"input": name, "suggestions": _suggest("HERBS", name)})
    for name in state["meds_in"]:
        hit = repo.find_drug(name)
        if hit:
            classes = repo.drug_classes(hit[0])
            drugs.append({"input": name, "id": hit[0], "name": hit[1], "classes": classes})
        else:
            unknown["drugs"].append({"input": name, "suggestions": _suggest("DRUGS", name)})
    state.update(herbs=herbs, drugs=drugs, unknown=unknown)
    return state

def _suggest(table, name):
    near = repo.nearest(table, embeddings.embed_query(name), 3)
    limit = config.SUGGEST_MAX_DIST or getattr(embeddings, "suggest_max_dist", 0.5)
    return [{"name": n, "distance": round(d, 3)} for n, d in near if d <= limit][:2]

def interactions(state):
    findings = []
    all_class_ids = sorted({cid for d in state["drugs"] for cid, _ in d["classes"]})
    for herb in state["herbs"]:
        rows = repo.rules(herb["id"], all_class_ids)
        by_rule: dict[int, dict] = {}
        for r in rows:
            by_rule.setdefault(r["rule_id"], {**r, "classes": []})["classes"].append(r["class_name"])
        for drug in state["drugs"]:
            drug_class_names = {n for _, n in drug["classes"]}
            for r in by_rule.values():
                matched = sorted(set(r["classes"]) & drug_class_names)
                if matched:
                    findings.append(
                        {
                            "herb": herb["name"],
                            "drug": drug["name"],
                            "matched_classes": matched,
                            "attention": r["attention"],
                            "evidence_type": r["evidence"],
                            "summary": r["summary"],
                            "mechanism": r["mechanism"],
                            "confidence": confidence(r["evidence"]),
                            "rule_id": r["rule_id"],
                            "evidence_passages": [],
                        }
                    )
    findings.sort(key=lambda f: (-ATT_RANK[f["attention"]], f["herb"], f["drug"]))
    state["findings"] = findings
    return state

def evidence(state):
    cache: dict[str, list] = {}
    for f in state["findings"]:
        if f["herb"] not in cache:
            q = f"{f['herb']} interaction with {', '.join(d['name'] for d in state['drugs'])}"
            cache[f["herb"]] = repo.evidence(q, f["herb"], k=3)
        f["evidence_passages"] = [p for p in cache[f["herb"]] if p["rule_id"] == f["rule_id"]][:1] or cache[f["herb"]][:1]
    return state

def gate(state):
    reasons = []
    if not state["herbs_in"] or not state["meds_in"]:
        reasons.append("Enter at least one herb and one medication.")
    if state["unknown"]["herbs"]:
        reasons.append("Herb not in the reference set: " + ", ".join(u["input"] for u in state["unknown"]["herbs"]))
    if state["unknown"]["drugs"]:
        reasons.append("Medication not identified: " + ", ".join(u["input"] for u in state["unknown"]["drugs"]))
    weak = [f for f in state["findings"] if f["confidence"] < CONFIDENCE_GATE]
    if weak:
        reasons.append(f"{len(weak)} finding(s) rest on weak or mixed evidence (confidence below {CONFIDENCE_GATE}).")
    state["reasons"] = reasons
    state["status"] = "needs_review" if reasons else "delivered"
```

```
        state["max_attention"] = max((f["attention"] for f in state["findings"]), key=lambda a: ATT_RANK[a], default="none")
        state["min_confidence"] = min((f["confidence"] for f in state["findings"]), default=None)
        return state

    def explain(state):
        lines = [
            f"{f['herb'][:1].upper() + f['herb'][1:]} + {f['drug']} [{f['attention'].upper()}]: {f['summary']} ({f['mechanism']})"
            for f in state["findings"]
        ]
        if not lines and state["status"] == "delivered":
            lines = ["No interaction found in this reference set. That is not evidence that the combination is safe."]
        if not lines and state["status"] == "needs_review":
            lines = ["The check could not be completed because some names were not recognized. See the reasons above. No result he
        text = "\n".join(lines)
        if explainer and state["findings"]:
            try:
                text = explainer(state["findings"]) or text
                state["explained_by"] = "llm"
            except Exception as exc:  # LLM is optional; never fail the request because of it
                state["explained_by"] = f"template (llm error: {exc.__class__.__name__})"
        else:
            state["explained_by"] = "template"
        state["explanation"] = text
        return state

    def audit(state):
        digest_src = json.dumps(sorted((f["herb"], f["drug"], f["attention"]) for f in state["findings"]))
        digest = hashlib.sha256(digest_src.encode()).hexdigest()[:16].upper()
        try:
            repo.log_check(
                len(state["herbs_in"]), len(state["meds_in"]), len(state["findings"]),
                state["max_attention"], state["status"], digest,
            )
            state["audit_logged"] = True
        except Exception:
            state["audit_logged"] = False
        return state

    def finish(state):
        return {
            "status": state["status"],
            "reasons": state["reasons"],
            "max_attention": state["max_attention"],
            "min_confidence": state["min_confidence"],
            "findings": state["findings"],
            "unrecognized": state["unknown"],
            "recognized": {
                "herbs": [h["name"] for h in state["herbs"]],
                "medications": [{"name": d["name"], "classes": [n for _, n in d["classes"]]} for d in state["drugs"]],
            },
            "explanation": state["explanation"],
            "explained_by": state["explained_by"],
            "disclaimer": DISCLAIMER,
            "db2": {
                "version": repo.version(),
                "embeddings": getattr(embeddings, "name", "custom"),
                "timings_ms": state["timings_ms"],
                "audit_logged": state["audit_logged"],
            },
        }

    def init(payload):
        return {**payload, "timings_ms": {}}

    steps = [
        RunnableLambda(init, name="init"),
        _stage("normalize", normalize),
        _stage("resolve_db2", resolve),
        _stage("interactions_db2_sql", interactions),
        _stage("evidence_db2_vector", evidence),
        _stage("gate", gate),
        _stage("explain", explain),
        _stage("audit_db2", audit),
        RunnableLambda(finish, name="finish"),
    ]
    chain = steps[0]
    for s in steps[1:]:
        chain = chain | s
    return chain
def make_explainer():
    """Optional grounded LLM summary through any OpenAI-compatible endpoint. Returns None if not configured."""
```

```
if not (config.LLM_API_KEY and config.LLM_MODEL):
    return None
from langchain_openai import ChatOpenAI  # optional dependency

llm = ChatOpenAI(model=config.LLM_MODEL, api_key=config.LLM_API_KEY, base_url=config.LLM_BASE_URL or None, temperature=0)

def explain(findings: list[dict]) -> str:
    facts = "\n".join(
        f"- {f['herb']} + {f['drug']} ({f['attention']} attention, evidence: {f['evidence_type']}): {f['summary']} Mechanism:
        for f in findings
    )
    prompt = (
        "You are a careful pharmacy reference assistant. Using ONLY the facts below, write a plain-language "
        "summary in at most 4 sentences. Do not add facts. Never say a combination is safe. End with: "
        "'Please ask a pharmacist or clinician.'\n\nFacts:\n" + facts
    )
    return llm.invoke(prompt).content

return explain
```

app/ingest.py

```
"""Create the Db2 schema and load the reference data plus embeddings.

    python -m app.ingest            # create + load (fails if tables already exist)
    python -m app.ingest --reset    # drop everything first, then create + load
"""
from __future__ import annotations

import json
import re
import sys
from pathlib import Path

import ibm_db_dbi

from . import config
from .embeddings import get_embeddings
from .repo import Db2Repo, dsn, vec_literal

ROOT = Path(__file__).resolve().parent.parent
TABLES_DROP_ORDER = [
    "V_RULE_OVERVIEW", "CHECK_LOG", "RULE_CLASSES", "INTERACTION_RULES", "DRUG_CLASS_MEMBERS",
    "DRUGS", "DRUG_CLASSES", "HERB_ALIASES", "HERBS", config.EVIDENCE_TABLE,
]


def split_sql(text: str) -> list[str]:
    text = "\n".join(l for l in text.splitlines() if not l.strip().startswith("--"))
    return [s.strip() for s in re.split(r";\s*\n", text + "\n") if s.strip()]


def drop_all(cur) -> None:
    for name in TABLES_DROP_ORDER:
        kind = "VIEW" if name.startswith("V_") else "TABLE"
        try:
            cur.execute(f"DROP {kind} {name}")
            print(f"  dropped {name}")
        except ibm_db_dbi.Error as exc:
            if "SQL0204N" not in str(exc):
                raise


def main(reset: bool = False) -> None:
    kb = json.loads((ROOT / "data" / "interaction_kb.json").read_text())
    emb = get_embeddings()
    print(f"Embeddings: {getattr(emb, 'name', 'custom')}")
    conn = ibm_db_dbi.connect(dsn(), "", "")
    cur = conn.cursor()
    if reset:
        print("Dropping existing objects...")
        drop_all(cur)
        conn.commit()

    print("Creating schema...")
    for stmt in split_sql((ROOT / "sql" / "schema.sql").read_text()):
        cur.execute(stmt)
    conn.commit()

    # ids are assigned here so every row can be inserted with plain executemany
    class_ids = {name: i for i, name in enumerate(kb["classes"], start=1)}
    drug_names = sorted({d for members in kb["classes"].values() for d in members})
    drug_ids = {d: i for i, d in enumerate(drug_names, start=1)}
    herb_names = list(kb["herbs"].keys())
    herb_ids = {h: i for i, h in enumerate(herb_names, start=1)}

    cur.executemany("INSERT INTO DRUG_CLASSES (CLASS_ID, CLASS_NAME) VALUES (?, ?)", [(i, n) for n, i in class_ids.items()])
    cur.executemany("INSERT INTO HERBS (HERB_ID, NAME) VALUES (?, ?)", [(i, n) for n, i in herb_ids.items()])
    cur.executemany("INSERT INTO DRUGS (DRUG_ID, NAME) VALUES (?, ?)", [(i, n) for n, i in drug_ids.items()])
    cur.executemany(
        "INSERT INTO HERB_ALIASES (HERB_ID, ALIAS) VALUES (?, ?)",
        sorted({(herb_ids[h], a.lower()) for h, v in kb["herbs"].items() for a in v.get("aliases", []) if a.lower() != h}),
    )
    cur.executemany(
        "INSERT INTO DRUG_CLASS_MEMBERS (DRUG_ID, CLASS_ID) VALUES (?, ?)",
        [(drug_ids[d], class_ids[c]) for c, members in kb["classes"].items() for d in members],
    )

    rule_rows, rule_class_rows, passages, metas = [], [], [], []
    rid = 0
```

```
    for herb, info in kb["herbs"].items():
        for r in info["rules"]:
            rid += 1
            rule_rows.append((rid, herb_ids[herb], r["attention"], r["evidence"], r["summary"], r["mechanism"]))
            rule_class_rows += [(rid, class_ids[c]) for c in r["classes"]]
            passages.append(
                f"{herb}: {r['summary']} Mechanism: {r['mechanism']} Applies to drug classes: "
                f"{', '.join(r['classes'])}. Evidence: {r['evidence']}."
            )
            metas.append({"herb": herb, "rule_id": rid, "attention": r["attention"], "classes": r["classes"]})
    cur.executemany(
        "INSERT INTO INTERACTION_RULES (RULE_ID, HERB_ID, ATTENTION, EVIDENCE, SUMMARY, MECHANISM) VALUES (?, ?, ?, ?, ?, ?)",
        rule_rows,
    )
    cur.executemany("INSERT INTO RULE_CLASSES (RULE_ID, CLASS_ID) VALUES (?, ?)", rule_class_rows)
    conn.commit()
    print(f"Loaded {len(herb_names)} herbs, {len(drug_names)} drugs, {len(class_ids)} classes, {len(rule_rows)} rules")

    print("Embedding herbs and drugs (stored in Db2 VECTOR columns)...")
    herb_texts = list(herb_names)  # names only: best for typo / "did you mean" matching
    cur.executemany(
        f"UPDATE HERBS SET EMBEDDING = VECTOR(?, {config.EMBED_DIM}, FLOAT32) WHERE HERB_ID = ?",
        [(vec_literal(v), herb_ids[h]) for h, v in zip(herb_names, emb.embed_documents(herb_texts))],
    )
    cur.executemany(
        f"UPDATE DRUGS SET EMBEDDING = VECTOR(?, {config.EMBED_DIM}, FLOAT32) WHERE DRUG_ID = ?",
        [(vec_literal(v), drug_ids[d]) for d, v in zip(drug_names, emb.embed_documents(drug_names))],
    )
    conn.commit()

    print("Indexing evidence passages with LangChain DB2VS...")
    repo = Db2Repo(embeddings=emb)
    repo._conn = conn  # reuse this connection
    store = repo._vector_store()
    store.add_texts(passages, metadatas=metas, ids=[f"rule-{m['rule_id']}" for m in metas])
    print(f"Indexed {len(passages)} passages in {config.EVIDENCE_TABLE}")

    cur.execute("SELECT COUNT(*) FROM V_RULE_OVERVIEW")
    print("V_RULE_OVERVIEW rows:", cur.fetchone()[0])
    print("Done.")


if __name__ == "__main__":
    main(reset="--reset" in sys.argv)
```

app/api.py

```
"""FastAPI service: POST /api/check, GET /api/health, GET /api/stats, and the demo UI at /."""
from __future__ import annotations

from pathlib import Path

from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import FileResponse, JSONResponse

from . import config
from .chain import build_chain, make_explainer
from .embeddings import get_embeddings
from .repo import Db2Repo

STATIC = Path(__file__).parent / "static"

app = FastAPI(title="Uday Shankar HerbCheck on IBM Db2", version="1.0.0")
_state: dict = {}


def get_chain():
    if "chain" not in _state:
        emb = get_embeddings()
        repo = Db2Repo(embeddings=emb)
        _state.update(repo=repo, emb=emb, chain=build_chain(repo, emb, make_explainer()))
    return _state["chain"]


def _validate(body: dict) -> dict:
    out = {}
    for key, limit in (("medications", config.MAX_MEDICATIONS), ("herbs", config.MAX_HERBS)):
        items = body.get(key)
        if not isinstance(items, list) or not items:
            raise HTTPException(422, f"'{key}' must be a non-empty list")
        if len(items) > limit:
            raise HTTPException(422, f"at most {limit} {key} per request")
        for it in items:
            if not isinstance(it, str) or not it.strip() or len(it) > config.MAX_NAME_LEN:
                raise HTTPException(422, f"each entry in '{key}' must be text of 1-{config.MAX_NAME_LEN} characters")
        out[key] = items
    return out


@app.post("/api/check")
async def check(request: Request):
    try:
        body = await request.json()
    except Exception:
        raise HTTPException(400, "invalid JSON")
    if not isinstance(body, dict):
        raise HTTPException(400, "JSON object expected")
    payload = _validate(body)
    try:
        chain = get_chain()
        return JSONResponse(chain.invoke(payload))
    except HTTPException:
        raise
    except Exception as exc:
        raise HTTPException(503, f"Db2 or pipeline error: {exc.__class__.__name__}: {str(exc)[:200]}")


@app.get("/api/health")
def health():
    try:
        get_chain()
        return {"status": "ok", "db2_version": _state["repo"].version(), "embeddings": getattr(_state["emb"], "name", "custom")}
    except Exception as exc:
        raise HTTPException(503, f"{exc.__class__.__name__}: {str(exc)[:200]}")


@app.get("/api/stats")
def stats():
    try:
        get_chain()
        return _state["repo"].stats()
    except Exception as exc:
        raise HTTPException(503, f"{exc.__class__.__name__}: {str(exc)[:200]}")
@app.get("/")
def index():
```

```
    return FileResponse(STATIC / "index.html")
```

tests/test_chain.py

```
import pytest

from app.chain import build_chain, clean, confidence
from app.embeddings import HashEmbeddings
from tests.memory_repo import MemoryRepo


@pytest.fixture()
def run():
    repo = MemoryRepo()
    chain = build_chain(repo, HashEmbeddings())

    def go(meds, herbs):
        return chain.invoke({"medications": meds, "herbs": herbs}), repo

    return go


def test_clean_normalizes():
    assert clean("  Warfarin!! ") == "warfarin"
    assert clean("St. John's   Wort") == "st. john's wort"


def test_confidence_levels():
    assert confidence("pharmacokinetic studies") == 0.93
    assert confidence("case reports") == 0.85
    assert confidence("theoretical") == 0.60
    assert confidence("case reports; mixed trials") <= 0.78


def test_warfarin_ginkgo_found_and_logged(run):
    res, repo = run(["Warfarin"], ["Ginkgo"])
    assert res["findings"], "expected at least one finding"
    assert {f["herb"] for f in res["findings"]} == {"ginkgo"}
    assert res["max_attention"] in {"moderate", "high"}
    assert res["findings"][0]["evidence_passages"]
    assert len(repo.logged) == 1 and repo.logged[0][-1]  # digest only, no raw input


def test_alias_resolves(run):
    res, _ = run(["warfarin"], ["gingko"])
    assert res["recognized"]["herbs"] == ["ginkgo"]


def test_typo_suggests_but_never_auto_accepts(run):
    res, _ = run(["warfarin"], ["tumeric"])
    assert res["status"] == "needs_review" and not res["findings"]
    assert res["unrecognized"]["herbs"][0]["suggestions"][0]["name"] == "turmeric"


def test_sertraline_st_johns_wort_high(run):
    res, _ = run(["sertraline"], ["st john's wort"])
    assert res["max_attention"] == "high"
    assert res["findings"][0]["attention"] == "high"


def test_unknown_herb_needs_review_with_suggestion(run):
    res, _ = run(["metformin"], ["ashwaghanda"])
    assert res["status"] == "needs_review"
    sug = res["unrecognized"]["herbs"][0]["suggestions"]
    assert sug and sug[0]["name"] == "ashwagandha"
    assert not res["findings"]  # a suggestion is never auto-accepted


def test_unknown_drug_needs_review(run):
    res, _ = run(["notarealdrugxyz"], ["ginkgo"])
    assert res["status"] == "needs_review"
    assert res["unrecognized"]["drugs"][0]["input"] == "notarealdrugxyz"


def test_no_findings_is_not_called_safe(run):
    res, _ = run(["metformin"], ["valerian"])
    assert "not evidence" in res["explanation"].lower() or res["findings"]


def test_duplicates_collapsed(run):
    res, _ = run(["warfarin", "WARFARIN"], ["ginkgo", "ginkgo "])
    assert len({(f["herb"], f["drug"], f["rule_id"]) for f in res["findings"]}) == len(res["findings"])
```

tests/test_kb.py

```
import json
from pathlib import Path

from app.embeddings import DIM, HashEmbeddings

KB = json.loads((Path(__file__).resolve().parent.parent / "data" / "interaction_kb.json").read_text())


def test_every_rule_class_exists():
    for herb, v in KB["herbs"].items():
        for r in v["rules"]:
            assert set(r["classes"]) <= set(KB["classes"]), (herb, r["classes"])
            assert r["attention"] in {"high", "moderate", "low"}
            assert len(r["summary"]) <= 600 and len(r["mechanism"]) <= 300 and len(r["evidence"]) <= 120


def test_names_fit_columns():
    assert all(len(h) <= 80 for h in KB["herbs"])
    assert all(len(d) <= 80 for m in KB["classes"].values() for d in m)
    assert all(len(c) <= 40 for c in KB["classes"])


def test_hash_embeddings_dimension_and_norm():
    v = HashEmbeddings().embed_query("ginkgo biloba")
    assert len(v) == DIM
    assert abs(sum(x * x for x in v) - 1) < 1e-6


def test_schema_statements_split():
    from app.ingest import split_sql

    stmts = split_sql((Path(__file__).resolve().parent.parent / "sql" / "schema.sql").read_text())
    assert len(stmts) == 10 and stmts[0].startswith("CREATE TABLE HERBS") and stmts[-1].startswith("CREATE VIEW")
```

tests/test_api.py

```
from fastapi.testclient import TestClient

from app import api
from app.chain import build_chain
from app.embeddings import HashEmbeddings
from tests.memory_repo import MemoryRepo

repo = MemoryRepo()
emb = HashEmbeddings()
api._state.update(repo=repo, emb=emb, chain=build_chain(repo, emb))
client = TestClient(api.app)


def test_check_ok():
    r = client.post("/api/check", json={"medications": ["warfarin"], "herbs": ["ginkgo"]})
    assert r.status_code == 200 and r.json()["findings"]


def test_validation_errors():
    assert client.post("/api/check", json={"medications": [], "herbs": ["ginkgo"]}).status_code == 422
    assert client.post("/api/check", json={"medications": ["x" * 61], "herbs": ["ginkgo"]}).status_code == 422
    assert client.post("/api/check", json={"medications": ["a"] * 7, "herbs": ["ginkgo"]}).status_code == 422
    assert client.post("/api/check", content=b"nope", headers={"content-type": "application/json"}).status_code == 400


def test_index_served():
    r = client.get("/")
    assert r.status_code == 200 and "HerbCheck" in r.text
```

scripts/setup_env.sh

```
#!/usr/bin/env bash
# Creates .env with a random Db2 password, so there is no placeholder to forget.
# Safe to re-run: it never overwrites an existing .env.
set -eu
cd "$(dirname "$0")/.."
if [ -f .env ]; then echo ".env already exists; leaving it alone."; exit 0; fi
PW="$(LC_ALL=C tr -dc 'A-Za-z0-9' </dev/urandom | head -c 20)"
sed "s/__GENERATED__/$PW/g" .env.example > .env
chmod 600 .env
echo "Created .env with a random Db2 password (not committed: .env is gitignored)."
```

scripts/smoke_test.sh

```
#!/usr/bin/env bash
# Usage: scripts/smoke_test.sh [base_url]   (default http://localhost:8000)
set -u
B="${1:-http://localhost:8000}"
echo "== health";  curl -s "$B/api/health"; echo
echo "== stats";   curl -s "$B/api/stats"; echo
echo "== warfarin + ginkgo"
curl -s -X POST "$B/api/check" -H 'Content-Type: application/json' -d '{"medications":["warfarin"],"herbs":["ginkgo"]}' | head -c
echo "== typo (tumeric) -> suggestion, needs_review"
curl -s -X POST "$B/api/check" -H 'Content-Type: application/json' -d '{"medications":["warfarin"],"herbs":["tumeric"]}' | head -c
```

docker-compose.yml

```
# IBM Db2 Community Edition (free, no card). Needs Docker and about 4 GB RAM.
services:
  db2:
    image: icr.io/db2_community/db2
    container_name: herbcheck-db2
    privileged: true
    ports:
      - "50000:50000"
    environment:
      LICENSE: accept
      DB2INSTANCE: db2inst1
      DB2INST1_PASSWORD: ${DB2INST1_PASSWORD:?set DB2INST1_PASSWORD in .env}
      DBNAME: herbdb
    volumes:
      - db2data:/database
    healthcheck:
      test: ["CMD-SHELL", "su - db2inst1 -c 'db2 connect to herbdb' > /dev/null 2>&1"]
      interval: 20s
      timeout: 15s
      retries: 40
      start_period: 120s
volumes:
  db2data:
```